

第二次编程实践课

软件工程与计算 I · 2026 · 南京大学

| Week 2 · M2 循环与分支 · 提示词工程

今日安排

知识讲解

Java 循环与分支

if/else · switch · for · while · break/continue

AI实践 M2 提示词工程

什么是好的 Prompt? 如何精确描述需求

大作业 大作业答疑

进度确认、选题问答

当堂练习

练习 共 2 题，独立完成

- 练习 1: 读懂循环与分支的执行轨迹
- 练习 2: 用循环解决实际问题

目标: 建立对控制流的精确直觉, 能追踪每一步状态

知识讲解

Java 循环与分支 · AI4Coding M2 提示词工程 · 大作业

变量与数据类型

基本数据类型 (Primitive)

类型	位数	示例值
int	32	0 , 42 , -1
long	64	100L
double	64	3.14 , 1.5e-3
float	32	3.14f
boolean	—	true , false
char	16	'A' , '\n'

声明与赋值

```
int count = 0;           // 声明同时初始化
double pi, r, area;      // 同类型可连续声明
pi = 3.1416;             // 赋数值给变量
area = pi * r * r;       // 赋表达式的值
```

常量 (final)

```
// 全大写 + 下划线, 声明后不可修改
static final int MAX_SIZE = 100;
static final double PI = 3.14159;
```

整数除法 vs 浮点除法

```
int a = 3, b = 2;
System.out.println(a / b);           // 1 (截断!)
System.out.println(a / (double)b);  // 1.5

// 常见陷阱: 求平均值
int sum = 3;
int avg = sum / 2;                   // ✗ 结果为 1, 精度丢失
double avg2 = sum / 2.0;            // ✓ 结果为 1.5
```

规则: 两个操作数都是整数 → 整数除法 (截断小数);
任意一个是浮点数 → 浮点除法

变量 — var 类型推断 (1/2)

什么是 var?

Java 10 引入的**局部变量类型推断**:
编译器根据右侧初始值自动推断类型, 无需显式写出类型名。

```
// 传统写法
int count = 42;
String name = "Alice";
double price = 9.99;

// 用 var 等价写法
var count = 42;           // 推断为 int
var name = "Alice";       // 推断为 String
var price = 9.99;         // 推断为 double
```

var 只是少写一遍类型, 变量依然是静态强类型!

var 的限制

场景	可用?
局部变量 (有初始化)	✓
for 循环变量	✓
方法参数	✗
返回值类型	✗
类的字段	✗
无初始值的声明	✗

```
// ✗ 以下写法均不合法
var x;                               // 无初始化
void foo(var s) { }                  // 方法参数
var getVal() { return 1; } // 返回值类型
```

变量 — `var` 类型推断 (2/2) 何时用、何时不用

✓ 推荐使用

```
// 泛型类型名冗长时
var list = new ArrayList<String>();
var map = new HashMap<String, List<Integer>>();

// for-each 循环变量
for (var entry : map.entrySet()) {
    System.out.println(entry.getKey());
}

// 局部临时变量
var sc = new Scanner(System.in);
var line = sc.nextLine();
```

✗ 不推荐滥用

```
// 右侧看不出类型，可读性下降
var x = compute();    // x 是什么类型？
var result = process(data); // 不清晰

// 字面量类型可能引起歧义
var y = 0;            // int? long?
var z = 1.0;          // double? float?
```

原则：右侧表达式能一眼看出类型时用 `var`，
否则保留显式类型，让代码更易读。

运算符与表达式 (1/2) — 算术与赋值

算术运算符

运算符	含义	示例
<code>+</code> <code>-</code> <code>*</code> <code>/</code>	四则运算	<code>3 + 4</code> → <code>7</code>
<code>%</code>	取余	<code>10 % 3</code> → <code>1</code>

优先级: `*` `/` `%` 高于 `+` `-`; 括号 `()` 可改变运算顺序

赋值运算符

```
int x = 10;
x += 3;    // x = x + 3 → 13
x -= 1;    // x = x - 1 → 12
x *= 2;    // x = x * 2 → 24
x /= 4;    // x = x / 4 → 6
x %= 4;    // x = x % 4 → 2
```

一元运算符

```
int x = 5;
int a = x++; // a=5, x=6 (后置: 先用后加)
int b = ++x; // b=7, x=7 (前置: 先加后用)
x--;         // x=6
boolean ok = !true; // false (逻辑非)
```

后置 `x++` vs 前置 `++x`:
单独成语句时无区别; 作为表达式时取值不同

表达式与语句

```
// 表达式: 求值得到一个值
3 + 4           // int 值 7
x > 0           // boolean 值 true
Math.abs(-5)    // int 值 5

// 表达式 + 分号 = 语句
count++;        // 语句
sum = a + b;    // 语句
```

运算符与表达式 (2/2) — 关系与逻辑

关系运算符 (结果为 boolean)

```
int a = 5, b = 3;
a == b    // false (注意: == 不是赋值 =)
a != b    // true
a > b     // true
a >= b    // true
a < b     // false
a <= b    // false
```

对对象 (如 String) 比较值内容要用 `.equals()`,
`==` 只比较引用地址!

逻辑运算符

```
boolean p = true, q = false;
p && q    // false (AND: 两者都真才为 true)
p || q    // true  (OR: 任一为真即为 true)
!p        // false (NOT: 取反)
```

短路求值 (Short-circuit)

```
// && 左侧为 false → 右侧不执行
if (arr != null && arr.length > 0) {
    // arr==null 时 arr.length 不执行, 避免空指针
}

// || 左侧为 true → 右侧不执行
if (cache != null || loadFromDisk()) {
    // cache 非空时 loadFromDisk() 不会被调用
}
```

利用短路求值做安全检查是 Java 的惯用法

类型转换（1/2） — 自动拓宽 vs 强制转型

自动拓宽（Widening）

小范围 → 大范围，编译器**自动**完成，无精度损失

byte → short → int → long → float → double

```
int i = 42;
long l = i;           // ✓ 自动: int → long
double d = i;         // ✓ 自动: int → double
float f = 3;          // ✓ 自动: int → float
```

强制转型（Narrowing / Casting）

大范围 → 小范围，**必须显式写出**，可能丢失数据

```
double pi = 3.14159;
int n = (int) pi;      // → 3, 小数截断!

long big = 10_000_000_000L;
int x = (int) big;     // → 溢出 (结果不确定)!

double d = 9.99;
int i = (int) d;       // → 9, 不是四舍五入!
```

截断不是四舍五入: (int) 9.99 → 9

拓宽与转型对比

	自动拓宽	强制转型
语法	直接赋值	(目标类型) 值
数据损失	无	可能截断/溢出

类型转换 (2/2) — String 与 char 互转

String 与数值互转

```
// 数值 → String
int n = 42;
String s1 = String.valueOf(n); // "42"
String s2 = Integer.toString(n); // "42"
String s3 = n + ""; // "42" (隐式拼接)

// String → 数值
String s = "123";
int i = Integer.parseInt(s); // 123
double d = Double.parseDouble(s); // 123.0
long l = Long.parseLong(s); // 123L

// 格式错误会抛异常
Integer.parseInt("abc"); // ❌ NumberFormatException
```

char 与 int 互转

```
char c = 'A';
int code = c; // 65 (自动拓宽)
char back = (char) 66; // 'B' (强制转型)

// Unicode 字符计算
char next = (char) ('A' + 1); // 'B'
char lower = (char) ('A' + 32); // 'a' (大写转小写)
```

字符运算本质是整数运算, `'A' + 1` 的结果是 `int`

常用字符 Unicode 值

字符	值	字符	值
'0'	48	'A'	65
'9'	57	'Z'	90
'a'	97	'z'	122

运算符补充 (1/2) — 位运算符

位运算符

运算符	含义	示例 (二进制)
&	按位 AND	0b1010 & 0b1100 → 0b1000 (8)
	按位 OR	0b1010 0b1100 → 0b1110 (14)
^	按位 XOR	0b1010 ^ 0b1100 → 0b0110 (6)
~	按位取反	~1 → -2 (补码)
<<	左移	1 << 3 → 8 (相当于 $\times 2^3$)
>>	算术右移	-8 >> 1 → -4 (符号位保留)
>>>	逻辑右移	-1 >>> 1 → 2147483647 (高位补 0)

Java 独有 >>> (无符号右移)；C 对负数右移的行为是实现定义的

常见位运算技巧

```
// 判断奇偶 (比 % 2 快)
(n & 1) == 1 // 奇数
(n & 1) == 0 // 偶数

// 乘/除 2 的幂次
n << 1 // n × 2
n >> 1 // n ÷ 2 (向下取整)
n << 3 // n × 8

// 判断 n 是否为 2 的幂次
(n > 0) && (n & (n - 1)) == 0

// 取低 k 位 (掩码)
n & 0xFF // 取低 8 位
n & 0xF // 取低 4 位
```

位运算常见于：权限标志位、哈希计算、底层优化

运算符补充 (2/2) — 三元运算符 · instanceof

三元运算符 (Ternary)

```
// 语法: 条件 ? 值A : 值B
int a = 5, b = 3;
int max = (a > b) ? a : b;           // max = 5

String label = (score >= 60) ? "及格" : "不及格";

// 等价的 if-else
String label;
if (score >= 60) label = "及格";
else          label = "不及格";
```

适合简单赋值；嵌套使用会严重降低可读性

```
// ❌ 嵌套三元: 难以阅读
String grade = score >= 90 ? "优" : score >= 75 ? "良" : "其他";

// ✅ 改用 if-else 或 switch
```

instanceof 运算符

```
Object obj = "Hello";

// 传统写法: 判断后手动转型
if (obj instanceof String) {
    String s = (String) obj;
    System.out.println(s.length());
}

// Java 16+: 模式匹配, 合并判断与转型
if (obj instanceof String s) {
    System.out.println(s.length()); // s 直接可用
}

// 还可加条件
if (obj instanceof String s && s.length() > 3) {
    System.out.println("长字符串: " + s);
}
```

instanceof 在类型转型前做安全检查,
避免 ClassCastException

String 新特性 (1/2) — 文本块 Text Block (Java 15+)

为什么需要文本块?

// 旧写法: 拼接 + 转义, 难以阅读

```
String json = "{\n" +  
    "  \"name\": \"Alice\",\n" +  
    "  \"age\": 20\n" +  
    "}";
```

```
String sql = "SELECT id, name\n" +  
    "FROM users\n" +  
    "WHERE age > 18\n" +  
    "ORDER BY name";
```

文本块写法 (Java 15+)

用 `"""..."""`, 所见即所得, 无需 `\n` 和 `\`

```
String json = """  
    {  
        "name": "Alice",  
        "age": 20  
    }  
    """;
```

```
String sql = """  
    SELECT id, name  
    FROM users  
    WHERE age > 18  
    ORDER BY name  
    """;
```

- 缩进以结尾 `"""` 的列位置为基准自动去除
- 末尾自动包含换行; 结尾 `"""` 前移可去掉尾部换行
- 适合: JSON、SQL、HTML、正则表达式等多行内容

String 新特性 (2/3) — Java 11+ 新增方法

空白处理

```
// isBlank(): 空串或纯空白 → true (比 isEmpty() 更实用)
"".isBlank()           // true
"  ".isBlank()         // true
"  ".isEmpty()         // false ← 注意区别!

// strip(): 支持 Unicode 空白 (trim() 只处理 ASCII)
" hello ".strip()      // "hello"
" hello ".stripLeading() // "hello "
" hello ".stripTrailing() // " hello"

// trim() 的局限
"\u3000hello\u3000".trim() // 无效 (\u3000 是全角空格)
"\u3000hello\u3000".strip() // "hello" ✓
```

其他新方法

```
// repeat(n): 重复 n 次
"=-".repeat(20) // "=-=-=-=-...=-" (画分割线)
"0".repeat(5)   // "00000"

// lines(): 按行切分为 Stream (比 split("\n") 更健壮)
"line1\nline2\nline3"
    .lines()
    .forEach(System.out::println);

// formatted(): 相当于 String.format (Java 15+)
"Hello, %s! You are %d years old."
    .formatted("Alice", 20);
// → "Hello, Alice! You are 20 years old."
```

String 新特性 (3/3) — Java 11+ 方法速查

方法	作用	对比旧用法	Java 版本
<code>isBlank()</code>	是否为空或纯空白	比 <code>isEmpty()</code> 更严格，识别空白字符	11
<code>strip()</code>	去首尾空白 (Unicode)	取代 <code>trim()</code> ，支持全角空格等 Unicode 空白	11
<code>stripLeading()</code>	去开头空白	<code>trim()</code> 无此精细控制	11
<code>stripTrailing()</code>	去结尾空白	<code>trim()</code> 无此精细控制	11
<code>repeat(n)</code>	重复 n 次	取代手动 <code>StringBuilder</code> 循环拼接	11
<code>lines()</code>	按行切分为 Stream	比 <code>split("\\n")</code> 更健壮 (处理 <code>\r\n</code>)	11
<code>indent(n)</code>	每行缩进 n 个空格	新增，文本块排版用	12
<code>formatted(args)</code>	格式化字符串	等价于 <code>String.format(this, args)</code>	15
<code>translateEscapes()</code>	解释转义序列	将 <code>\\n</code> 转为真正换行符	15

记忆技巧: Java 11 一次补齐了"空白处理三件套" (`isBlank` / `strip` / `repeat` / `lines`);
Java 15 随文本块同步加入了 `formatted` 和 `translateEscapes`

分支语句 — if / else if / else

基本结构

```
if (条件1) {  
    // 条件1 为 true 时执行  
} else if (条件2) {  
    // 条件1 false, 条件2 true 时执行  
} else {  
    // 以上均不满足时执行  
}
```

成绩分级示例

```
int score = 85;  
if (score >= 90) {  
    System.out.println("优秀");  
} else if (score >= 75) {  
    System.out.println("良好");    // ← 执行这里  
} else if (score >= 60) {  
    System.out.println("及格");  
} else {  
    System.out.println("不及格");  
}
```

常见陷阱

```
// ❌ 陷阱1: 条件顺序错误  
if (score >= 60) {  
    System.out.println("及格");    // 90分也会进这里!  
} else if (score >= 90) {  
    System.out.println("优秀");    // 永远不会到达  
}
```

```
// ❌ 陷阱2: == 比较浮点数  
double x = 0.1 + 0.2;  
if (x == 0.3) { ... }    // false! 浮点精度问题
```

```
// ✅ 正确做法  
if (Math.abs(x - 0.3) < 1e-9) { ... }
```

只有一个分支会执行：Java 从上到下依次检查，第一个为 `true` 的分支执行完后直接跳出

分支语句 — switch (1/2) 传统写法

基本语法

```
int day = 3;
switch (day) {
    case 1:
        System.out.println("周一");
        break;           // ← 必须! 否则贯穿
    case 2:
        System.out.println("周二");
        break;
    case 3:
    case 4:
    case 5:
        System.out.println("工作日"); // 3/4/5 共享
        break;
    default:
        System.out.println("周末");
}
```

fall-through 陷阱

```
// ✗ 忘写 break: day=1 会同时输出"周一"和"周二"!
switch (day) {
    case 1:
        System.out.println("周一");
        // 没有 break, 继续执行 case 2
    case 2:
        System.out.println("周二");
        break;
}
```

贯穿 (fall-through) 是 switch 经典 Bug 来源,
每个 case 末尾都要检查是否需要 `break`

分支语句 — switch (2/2) 现代写法

switch 表达式 (Java 14+, 推荐)

```
// 箭头语法: 无 fall-through, 无需 break
String result = switch (day) {
    case 1 -> "周一";
    case 2 -> "周二";
    case 3, 4, 5 -> "工作日";    // 多值合并
    default -> "周末";
};
System.out.println(result);
```

箭头 `->` 语法天然防止 fall-through, 代码更简洁, 推荐在 Java 14+ 项目中使用

switch 支持的类型

支持	不支持
<code>int</code> / <code>Integer</code>	<code>long</code> / <code>double</code>
<code>String</code>	<code>boolean</code>
<code>char</code> / <code>Character</code>	浮点类型
<code>enum</code>	

if-else vs switch 的选择

场景	推荐
值相等判断 (多个固定值)	<code>switch</code>
范围判断 (如 <code>score >= 90</code>)	<code>if-else</code>
复杂布尔条件	<code>if-else</code>

分支语句 — switch 模式匹配 (Java 21+)

类型模式 (Type Patterns)

Java 21 起 switch 可以直接对类型做匹配, 不再局限于值

```
// 旧写法: if-instanceof 链
if (obj instanceof Integer i) {
    System.out.println("整数: " + i);
} else if (obj instanceof String s) {
    System.out.println("字符串长度: " + s.length());
} else if (obj instanceof Double d) {
    System.out.println("浮点: " + d);
}

// Java 21+ switch 类型匹配 (更简洁)
switch (obj) {
    case Integer i -> System.out.println("整数: " + i);
    case String s -> System.out.println("字符串: " + s.length());
    case Double d -> System.out.println("浮点: " + d);
    default -> System.out.println("其他类型");
}
```

守卫模式 (Guarded Patterns)

在类型匹配后加 `when` 条件进一步细化

```
switch (obj) {
    case Integer i when i > 0 -> "正整数";
    case Integer i when i < 0 -> "负整数";
    case Integer i -> "零";
    case String s when s.isBlank() -> "空字符串";
    case String s -> "字符串: " + s;
    default -> "其他";
}
```

switch 中处理 null (Java 21+)

```
// 旧写法: 必须在 switch 外判断 null, 否则抛 NullPointerException
// Java 21+: 可直接在 case 中处理
switch (obj) {
    case null -> System.out.println("空值");
    case String s -> System.out.println("字符串");
    default -> System.out.println("其他");
}
```

循环语句 — for 循环 (1/2) 基本语法

基本结构与执行顺序

```
// for (① 初始化; ② 条件; ④ 更新)
for (int i = 0; i < 5; i++) {
    // ③ 循环体
    System.out.println(i);    // 输出 0 1 2 3 4
}
// 执行顺序: ① → ②③④ → ②③④ → ... → ②(false) 退出
```

关键：初始化 ① 只执行一次；
每轮先检查条件 ②，再执行体 ③，最后更新 ④

常见遍历模式

```
int[] arr = {10, 20, 30, 40};

// 正向遍历
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}

// 增强 for (for-each) — 只读遍历推荐
for (int x : arr) {
    System.out.println(x);
}

// 反向遍历
for (int i = arr.length - 1; i >= 0; i--) {
    System.out.println(arr[i]);
}
```

循环语句 — for 循环 (2/2) 常见陷阱

三种经典错误

```
// ✗ 无限循环: 条件永远 true
for (int i = 0; i >= 0; i++) { ... }

// ✗ 差一错误 (off-by-one)
for (int i = 0; i <= arr.length; i++) {
    System.out.println(arr[i]); // arr.length 时越界!
}

// ✓ 正确: < 而非 <=
for (int i = 0; i < arr.length; i++) { ... }
```

修改循环变量的危险

```
// ✗ 循环体内修改 i: 实际步长变成 2
for (int i = 0; i < 10; i++) {
    i++; // 容易引发难以发现的 Bug
}

// ✓ 如需步长 2, 在更新部分声明
for (int i = 0; i < 10; i += 2) { ... }
```

规律: 数组下标用 `i < arr.length` (严格小于),
计数 1 到 n 用 `i <= n` (小于等于)

循环语句 — while (1/2) 基本语法与模式

while — 先判断再执行

```
// 语法
while (条件) {
    // 条件为 true 时执行
}
```

```
// 示例：计数循环
int n = 1;
while (n <= 5) {
    System.out.println(n); // 输出 1 2 3 4 5
    n++;
}
```

条件在**每次执行前**检查；

条件一开始就为 false 时，循环体**一次都不执行**

while 的典型模式

```
// 模式1：累积计算
int sum = 0, i = 1;
while (i <= 100) {
    sum += i;
    i++;
} // sum = 5050
```

```
// 模式2：数字分解（逐位取出）
int num = 12345;
while (num > 0) {
    System.out.println(num % 10); // 5 4 3 2 1
    num /= 10;
}
```

```
// 模式3：读取输入直到哨兵值
Scanner sc = new Scanner(System.in);
int x = sc.nextInt();
while (x != -1) { // -1 为哨兵
    System.out.println(x);
    x = sc.nextInt();
}
```

循环语句 — do-while（2/2）与三种循环对比

do-while — 先执行再判断

```
// 语法
do {
    // 先执行，至少一次
} while (条件);

// 示例：输入验证（至少问一次）
int choice;
do {
    System.out.print("请选择(1-3): ");
    choice = sc.nextInt();
} while (choice < 1 || choice > 3);

// 示例：菜单循环
do {
    printMenu();
    int op = sc.nextInt();
    handle(op);
} while (op != 0); // 0 退出
```

do-while 保证至少执行一次，适合"先做再判断要不要继续"的场景

三种循环的选择

场景	推荐
已知循环次数	for
条件控制，可能 0 次	while
至少执行一次	do-while

while vs do-while 的关键区别

```
int x = 10;

// while: 条件不满足，一次不执行
while (x < 5) {
    System.out.println("while"); // 不输出
}

// do-while: 先执行一次，再判断
do {
    System.out.println("do-while"); // 输出一次!
} while (x < 5);
```

break & continue — 精细控制循环

break — 立即退出循环

```
// 找到第一个偶数就停止
int[] nums = {1, 3, 7, 4, 9, 2};
int found = -1;
for (int n : nums) {
    if (n % 2 == 0) {
        found = n;
        break; // 跳出整个 for 循环
    }
}
System.out.println(found); // 4
```

continue — 跳过本次，继续下一次

```
// 只打印奇数
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue; // 偶数跳过
    System.out.print(i + " "); // 1 3 5 7 9
}
```

嵌套循环中的 break/continue

```
// ✗ 常见误解: break 只跳出最内层循环
outer: // 标签
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (i == 1 && j == 1) {
            break outer; // 跳出外层循环
        }
        System.out.println(i + "," + j);
    }
}
// 输出: 0,0 0,1 0,2 1,0 (然后全部退出)
```

带标签的 `break` / `continue` 可以跳出指定层级，但过度使用会降低代码可读性，考虑用方法封装

嵌套循环 (1/2) — 九九乘法表

代码

```
for (int i = 1; i <= 9; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.printf("%d×%d=%-3d", j, i, i * j);
    }
    System.out.println();
}
```

输出:

```
1×1=1
1×2=2  2×2=4
1×3=3  2×3=6  3×3=9
...
```

执行次数分析

外层: $i = 1, 2, \dots, 9$ (9次)
内层: $j = 1..i$ (随 i 变化)
总次数 = $1+2+\dots+9 = 45$ 次 ($n*(n+1)/2$)

相比暴力 $i \times j$ (81次), 上三角写法减少了近一半迭代, 这是利用对称性优化的基本思路

嵌套循环复杂度

结构	执行次数
for i=1..n; for j=1..n	n^2
for i=1..n; for j=1..i	$n(n+1)/2$
for i=1..n; for j=1.. $\sqrt{n}$	$n\sqrt{n}$

嵌套循环（2/2） — 坐标思维与图案打印

直角三角形

```
int n = 5;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

输出：

```
*
* *
* * *
* * * *
* * * * *
```

坐标思维

把 `i` 想成行，`j` 想成列，
内层条件 `j <= i` 决定"该格是否打印"

行 i	打印列 j	星号数
1	$j \leq 1$	1
2	$j \leq 2$	2
3	$j \leq 3$	3

练习：等腰三角形怎么改？

```
// 提示：先打印空格，再打印 *
// 第 i 行：(n-i) 个空格 + (2i-1) 个 *
```

综合示例 (1/2) — 统计成绩分布

```
import java.util.Scanner;

public class GradeCount {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int excellent = 0, good = 0, pass = 0, fail = 0;
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            int score = sc.nextInt();
            if (score >= 90)        excellent++;
            else if (score >= 75) good++;
            else if (score >= 60) pass++;
            else                    fail++;
        }
        System.out.printf(
            "优秀:%d 良好:%d 及格:%d 不及格:%d\n",
            excellent, good, pass, fail);
    }
}
```

注意: `if / else if` 链只会进入第一个满足条件的分支, 顺序从严到宽 (≥ 90 写前面), 确保每个区间互不重叠

综合示例（2/2） — 判断素数

```
public static boolean isPrime(int n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;    // 排除偶数

    // 只需检查到 √n
    for (int i = 3; i * i <= n; i += 2) {
        if (n % i == 0) return false;
    }
    return true;
}
```

算法优化思路

版本	检查范围	效率
暴力	2 到 n-1	O(n)
优化1	2 到 √n	O(√n)
优化2	奇数 + √n	O(√n/2)

两个核心技巧

- 提前返回：满足条件立刻 `return false`，无需等循环结束
- 循环条件 `i * i <= n`：避免浮点 `Math.sqrt()`，整数比较更安全

AI4Coding 实践 — M2 提示词工程

核心问题 同一个需求，为什么不同的 Prompt 得到天差地别的结果？

什么是提示词（Prompt）？

Prompt 是你给 AI 的全部上下文：
任务描述 + 约束条件 + 期望格式 + 示例

模糊 vs 精确的差距

模糊 Prompt：
"写个循环程序"

精确 Prompt：
"用 Java 写一个方法，接收 int 数组，
返回所有偶数之和。不使用流式 API，
只用 for 循环。参数可能为 null 或空数组，
需要处理这两种边界情况。"

AI 输出质量 = 你的 Prompt 质量

好 Prompt 的四个要素

要素	说明	示例
角色	告诉 AI 扮演什么专家	"你是一位 Java 教师"
任务	明确要做什么	"解释以下代码的每一行"
约束	限定范围和格式	"只用基础语法，不用 lambda"
示例	给出期望的输入输出	"输入：[1,2,3]，输出：6"

提示词技巧 (1/2) — 角色设定与任务拆解

角色设定 (Role Prompting)

"你是一位有 10 年经验的 Java 工程师，正在给大一学生讲解循环语句。请用简单的比喻解释 while 和 do-while 的区别，不要使用技术术语。"

角色设定让 AI 调整表达风格和知识深度

任务拆解 (Task Decomposition)

✗ 一次性要求太多：
"帮我写一个学生成绩管理系统"

✓ 分步骤拆解：
第1步: "设计一个 Student 类，包含姓名和成绩字段"
第2步: "写一个方法，接收 Student 数组，返回平均分"
第3步: "写一个方法，找出最高分的学生"

复杂任务拆小步，每步验证后再推进，AI 输出更准确，你也更容易发现问题

提示词技巧（2/2） — 示例驱动

Few-shot Prompting

"参考以下示例，按相同格式解释代码：

示例输入：

```
int x = 5;
```

示例解释：

- 声明整型变量 x
- 赋初值 5，存入内存

现在解释这段代码：

```
for (int i = 0; i < n; i++) {  
    sum += i;  
}
```

给 AI 看你期望的格式，它会照着做

三种技巧的适用场景

技巧	何时用
角色设定	需要特定风格或深度
任务拆解	复杂、多步骤任务
示例驱动	需要固定输出格式

组合使用效果最佳

角色设定 + 任务拆解 + 示例
= 高质量 Prompt 的完整结构

提示词技巧 — 追问与迭代 (1/2)

让 AI 解释自己的代码

Step 1: "用 Java 实现冒泡排序"

Step 2: "解释第 8 行为什么要用 `arr.length-1-i` 作为内层循环上界"

Step 3: "如果输入已经是有序数组, 这个实现会做多少次无效比较? 怎么优化?"

追问细节 比一次性要求完整代码更有效

要求 AI 找 Bug

"下面这段代码应该输出 1 到 10 的和 (55),
但实际输出了 54, 帮我找出错误:"

```
int sum = 0;
for (int i = 1; i < 10; i++) {
    sum += i;
}
System.out.println(sum);"
```

提供期望结果 vs 实际结果, 让 AI 聚焦在 diff 上,
比直接问"哪里有问题"效果更好

提示词技巧 — 追问与迭代 (2/2)

Chain of Thought — 让 AI 逐步推理

"请一步一步思考并回答：

一个数组 `arr = [3, 1, 4, 1, 5, 9]`，执行以下代码后输出什么？

```
int result = 0;
for (int i = 0; i < arr.length; i++) {
    if (arr[i] % 2 != 0) {
        result += arr[i];
    }
}
System.out.println(result);
```

请先列出每次循环的 `i`、`arr[i]`、`result` 值，再给出最终答案。"

要求"一步步推理"能显著提升 AI 的准确率——
这和你手动追踪执行轨迹的方法完全一致

AI4Coding 实践任务

任务 用不同质量的 Prompt，感受输出差距：

Step 1: 模糊 Prompt 实验

"写个 Java 循环"

- 记录 AI 生成了什么
- 它做了哪些你没说的假设?

Step 2: 精确 Prompt 实验

"用 Java 写一个方法，用 for 循环计算 1 到 n 的整数之和（n 由参数传入），要求处理 $n \leq 0$ 的情况并返回 0"

- 与 Step 1 的结果对比
- AI 是否处理了边界情况?

Step 3: 带角色 + 示例

"你是 Java 助教。参考以下示例格式，解释 break 语句的用途并给出两个不同场景的例子：
示例格式：【用途】... 【场景1】... 【场景2】..."

Step 4: 填表反思

问题	你的发现
哪个 Prompt 质量最高?	
AI 最容易在哪里出错?	
追问 vs 一次性要求，哪个更好?	
Prompt 写得好需要什么能力?	

大作业 — 用 Java 构建 Lambda 解释器 (1/2)

本学期大作业：逐步构建一个完整的 Lambda 演算解释器
共 9 个递进子项目，每次课布置 1~2 个

解释器流水线



9 个子项目概览

#	项目	核心内容
01	REPL	循环读取 + 回显
02	Lexer	字符串 → Token
03	AST	Token → 语法树
04	Eval	基础求值
05	Env	词法作用域
06	Control	闭包 + 条件
07	TCO	尾调用优化
08	Atom	动态求值 + 可变状态
09	Quote	元编程

大作业 — 用 Java 构建 Lambda 解释器 (2/2)

OOP 核心概念对应

OOP 概念	在本项目中的体现
继承	<code>LObject</code> 、 <code>Token</code> 子类层次
多态	<code>eval</code> 处理不同 <code>LObject</code> 子类
抽象类	<code>LObject</code> 、 <code>Token</code> 公共接口
封装	<code>Environment</code> 私有绑定表
组合	<code>LList</code> 组合 <code>LObject</code>
函数对象	<code>LFunction</code> 包装 Java Lambda
不可变性	<code>LInteger</code> 、 <code>LList</code> 不可变对象

技术栈

- 语言：Java
- 构建工具：Maven
- 测试框架：JUnit
- 包结构：`edu.nju.software.{token, stream, ast, eval}`

贯穿全学期的设计哲学

从最简单的 `while` 循环 (Project 01) 开始，每增加一个子项目，就是在一个真实软件上应用新的 OOP 概念。代码会随着课程进度持续演进，不是从零开始的一次性作业。

大作业 Project 01 — REPL (1/2) 任务描述

本次布置 截止时间：下次课前

任务描述

实现一个 Read-Eval-Print Loop (交互式循环):

- 循环读取用户每行输入
- 原样输出 (暂不解析, 只回显)
- 输入 `exit` 时退出程序

示例交互

```
> hello world
hello world
> (+ 1 2)
(+ 1 2)
> exit
```

为什么从 REPL 开始?

所有交互式语言 (Python、Node.js、Ruby) 底层都是一个 `while` 循环。
先搭好这个骨架, 后续 8 个子项目只需往里填内容。

REPL 的含义

字母	含义	本项目对应
R	Read (读取)	<code>sc.nextLine()</code>
E	Eval (求值)	暂时直接回显
P	Print (输出)	<code>println()</code>
L	Loop (循环)	<code>while (true)</code>

大作业 Project 01 — REPL (2/2) 实现要点

核心代码

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.print("> ");
            String line = sc.nextLine();
            if (line.equals("exit")) {
                break; // ← 本周核心: 循环 + 分支
            }
            System.out.println(line);
        }
    }
}
```

需要掌握的知识点

- `while (true)` + `break` 控制循环退出
- `String.equals()` 比较字符串 (不能用 `==`)
- `Scanner.nextLine()` 读取整行 (含空格)
- Maven 项目结构: 代码放 `src/main/java/`

思考

如果用 `while (!line.equals("exit"))` 能否改写?
两种写法有什么区别?
(提示: 想想 `line`
在循环开始前需要初始化为什么值)

大作业 Project 01 — 扩展要求 (1/2)

基础版（必须完成）

```
> hello
hello
> exit
(程序退出)
```

扩展 1：输出行号

```
[1] > hello
hello
[2] > world
world
[3] > exit
```

提示：在循环里维护一个计数器变量

扩展 2：统计会话信息

```
> hello
hello
> world
world
> exit
共处理 2 行输入，再见！
```

提示：在循环外定义 `count` 变量，
每次成功处理一行时 `count++`，
`break` 前输出统计信息

大作业 Project 01 — 扩展要求 (2/2) 提交规范

提交规范

```
project01/
├── pom.xml
├── src/
│   ├── main/java/
│   │   └── edu/nju/software/
│   │       └── Main.java
└── README.md      ← 说明你的扩展实现
```

自测方法

```
mvn compile
mvn exec:java -Dexec.mainClass="edu.nju.software.Main"
```

评分标准

要求	分值
基础 REPL 功能正常	60%
代码结构清晰，无冗余	20%
至少完成 1 项扩展	20%

用 AI 辅助写代码时，把你理解后的版本提交，不要直接复制粘贴

课后作业

必做

1. **完成今日Lab02+Homework02**（如未做完）
提交到 SEECODER-Coder，Lab截止 48 小时内，Homework截止下次上课前。
2. **Prompt 实验报告**
记录今日 AI 实践的 4 个 Prompt 及对比结果（一段话即可，不需要正式格式）
3. **用循环解决一个生活问题**
自选：统计、搜索、排序，提交代码

选做

- 用 AI 帮你写一段含 Bug 的循环代码，然后自己找出 Bug，再请 AI 验证你的分析
- 思考：`for` 和 `while` 可以互相转换吗？举例说明

预习

下周内容：方法与函数设计、递归初探

带着问题来：什么情况下应该把代码提取成一个方法？

当堂练习

当堂练习 · 独立完成 · 25 分钟

练习 1 — 读懂执行轨迹（阅读）

15 min 阅读以下代码，**手动追踪每一步**，再翻到下一页回答问题：

```
public class Trace {  
    public static void main(String[] args) {  
        int result = 0;  
        for (int i = 1; i <= 5; i++) {  
            if (i % 2 == 0) {  
                result += i * i;  
            } else {  
                result -= i;  
            }  
        }  
        System.out.println(result);  
    }  
}
```

从 `i = 1` 开始，逐行执行，记录每次循环结束时 `result` 的值

练习 1 — 读懂执行轨迹（答题）

① 填写执行轨迹表：

i	i % 2	执行分支	result
1			
2			
3			
4			
5			

② 最终输出是什么？

③ 如果把 `i <= 5` 改为 `i < 5`，结果如何变化？

④ 如果把 `result -= i` 改为 `result -= i * i`，输出是什么？

⑤ 用一句话描述这段程序的功能
(不要说"遍历循环"，要说做了什么计算)。

练习 2 — 编写循环程序

10 min

题目 A（必做）

输入一个正整数 `n`，输出从 1 到 `n` 中所有能被 3 整除但不能被 5 整除的数，每个数占一行，最后一行输出满足条件的数的个数。

示例 (`n = 20`):

```
3
6
9
12
18
共 5 个
```

提示：先想清楚条件，再写循环

题目 B（挑战）

输入一行包含若干个整数（空格分隔），输出其中的最大值和最小值，以及它们的差。若只有一个数，差为 0。

示例：

```
输入：5 3 8 1 9 2
输出：
最大值：9
最小值：1
差：8
```

提示：用 `Scanner` 或 `BufferedReader` 读取整行，用 `split(" ")` 拆分，注意初始化最大最小值

附录

有代码就得有测试！ · JUnit 入门

为什么要写测试？

没有测试的开发方式

```
// 写完代码，靠"看"来验证
public static int sum(int a, int b) {
    return a - b;    // ← Bug! 写成减法
}

// main 里随手跑一下
public static void main(String[] args) {
    System.out.println(sum(1, 2)); // 输出 -1
    // 看到 -1, 才发现写错了
}
```

问题：

- 需要手动运行、肉眼判断
- 改了代码后要重新验证所有功能
- Bug 越晚发现，修复成本越高

有测试的开发方式

```
@Test
public void testSum_正常() {
    assertEquals(3, sum(1, 2)); // 自动判断
}

@Test
public void testSum_负数() {
    assertEquals(-1, sum(-3, 2));
}
```

优势：

- 一键运行，自动报告通过 / 失败
- 代码修改后**立即回归验证**
- 测试即文档，清晰说明期望行为

有代码就得有测试：

写完一个方法，立刻写对应的测试用例

JUnit 4 — 基本用法

核心注解与断言

```
import org.junit.Test;
import static org.junit.Assert.*;

public class CalculatorTest {

    @Test                // 标记为测试方法
    public void testAdd() {
        // assertEquals(期望值, 实际值)
        assertEquals(5, Calculator.add(2, 3));
        assertEquals(0, Calculator.add(-1, 1));
    }

    @Test
    public void testDivide_整数除法() {
        assertEquals(2, Calculator.divide(7, 3));
    }
}
```

常用断言方法

断言	含义
assertEquals(exp, act)	值相等
assertTrue(condition)	条件为 true
assertFalse(condition)	条件为 false
assertNull(obj)	对象为 null
assertNotNull(obj)	对象非 null

Maven 依赖 (pom.xml)

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
```


JUnit 4 — 测试用例设计

黑盒测试三类边界

类别	示例 (isPrime)
正常值	isPrime(13) → true
边界值	isPrime(2), isPrime(1), isPrime(0)
异常值	isPrime(-5)

每个 `@Test` 只测一个场景，方法名说明测什么

```
@Test
public void test_2是最小素数() {
    assertTrue(MathUtil.isPrime(2));
}

@Test
public void test_1不是素数() {
    assertFalse(MathUtil.isPrime(1));
}
```

运行与结果

```
mvn test
```

全部通过（绿色）

```
Tests run: 4, Failures: 0
BUILD SUCCESS
```

有用例失败（红色）

```
test_负数 FAILED
expected:<-1> but was:<-5>
    at CalculatorTest.java:12
BUILD FAILURE
```

红色不是坏事——测试在帮你找 Bug，
比程序到用户那才出错要好得多

SEECODER-Coder系统 — 明测试与暗测试

明测试 随题目发布，学生可见

```
// ArrayAnalyzerTest.java
@Test
public void testCountAboveAvg_典型() {
    // avg=3.0, 4 和 5 大于均值
    assertEquals(2,
        ArrayAnalyzer.countAboveAverage(
            new int[]{1, 2, 3, 4, 5}));
}

@Test
public void testCountAboveAvg_空数组() {
    assertEquals(0,
        ArrayAnalyzer.countAboveAverage(new int[]{}));
}
```

帮助学生理解题意，验证基本正确性

暗测试 提交后运行，不提前公开

```
// PrivateArrayAnalyzerTest.java
@Test
public void test_非整数均值() {
    // [1,2] avg=1.5, 只有 2>1.5
    // 考察: avg 是否用 double 而非 int
    assertEquals(1,
        ArrayAnalyzer.countAboveAverage(
            new int[]{1, 2}));
}

@Test
public void test_含负数() {
    assertEquals(1,
        ArrayAnalyzer.countAboveAverage(
            new int[]{-1, 0, 1}));
}
```

覆盖边界与陷阱，检验代码真正的健壮性